

Physics-Structured Dynamics Models for Model-Based RL

Lagrangian formulation, network parameterization, and the controller

lagrangian-mbrl-franka

June 14, 2026

Abstract

We learn the forward dynamics of the 7-DoF Franka Emika Panda, $(\mathbf{q}, \dot{\mathbf{q}}, \boldsymbol{\tau}) \mapsto \ddot{\mathbf{q}}$, to use as the predictive model inside a model-based reinforcement-learning (MBRL) loop. This note states the Euler–Lagrange equations the dynamics obey, shows *exactly where the neural network enters* a Deep Lagrangian Network (DeLaN), lists the architectures used for the structured model and the unstructured baselines, fixes the statistical error convention used by the project, and specifies the controller that plans through the learned model.

1 Problem

A rigid-body manipulator with generalized coordinates $\mathbf{q} \in \mathbb{R}^n$ ($n = 7$ for the Franka) and applied joint torques $\boldsymbol{\tau} \in \mathbb{R}^n$ evolves according to the manipulator equation

$$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}, \quad (1)$$

Here $\mathbf{M}(\mathbf{q}) \succ 0$ is the symmetric positive-definite mass matrix. The vector $\mathbf{c}$ collects Coriolis and centrifugal terms, while $\mathbf{g}$ is the gravitational force. The learning target is the *forward dynamics*

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}(\boldsymbol{\tau} - \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{g}(\mathbf{q})). \quad (2)$$

2 Lagrangian mechanics

The dynamics (1) are the Euler–Lagrange equations of the Lagrangian $\mathcal{L} = T - V$, with kinetic energy $T = \frac{1}{2} \dot{\mathbf{q}}^\top \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$ and potential $V(\mathbf{q})$:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \boldsymbol{\tau}. \quad (3)$$

Carrying out the differentiation recovers (1) with $\mathbf{g}(\mathbf{q}) = \partial V / \partial \mathbf{q}$ and the Coriolis term expressed through derivatives of $\mathbf{M}$. Encoding this structure is the whole point: the true dynamics *lie in* this class, so a model constrained to it has a smaller effective hypothesis space and needs fewer samples to reach a given accuracy (the sample-complexity argument of `PROJECT_PLAN.md` §2).

3 Where the network goes: Deep Lagrangian Networks (DeLaN)

DeLaN [1] does *not* regress $\ddot{\mathbf{q}}$ directly. Instead, two neural networks parameterize the *energy*, and the dynamics follow by automatic differentiation of (3). Concretely, with parameters $\boldsymbol{\theta}$:

- a network outputs the entries of a lower-triangular $\mathbf{L}(\mathbf{q}; \boldsymbol{\theta})$, giving a guaranteed-SPD mass matrix

$$\mathbf{M}(\mathbf{q}; \boldsymbol{\theta}) = \mathbf{L}(\mathbf{q}; \boldsymbol{\theta}) \mathbf{L}(\mathbf{q}; \boldsymbol{\theta})^\top + \varepsilon \mathbf{I}, \quad \varepsilon > 0, \quad (4)$$

(the diagonal of $\mathbf{L}$ is passed through a shifted softplus so it stays strictly positive, which conditions the solves);

- a scalar network outputs the potential $V(\mathbf{q}; \boldsymbol{\theta})$.

The kinetic-plus-potential Lagrangian $\mathcal{L} = \frac{1}{2} \dot{\mathbf{q}}^\top \mathbf{M}(\mathbf{q}; \boldsymbol{\theta}) \dot{\mathbf{q}} - V(\mathbf{q}; \boldsymbol{\theta})$ is then differentiated through (3). Using the identity implemented in the code, the combined Coriolis-plus-gravity force is

$$\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \left(\frac{\partial^2 \mathcal{L}}{\partial \dot{\mathbf{q}} \partial \mathbf{q}} \right) \dot{\mathbf{q}} - \frac{\partial \mathcal{L}}{\partial \mathbf{q}}, \quad (5)$$

and the predicted acceleration follows from (2) by solving the SPD system $\mathbf{M} \ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{c} - \mathbf{g}$ (a Cholesky solve). Because (5) takes *second* derivatives of $\mathcal{L}$, the network activations must be twice continuously differentiable (C^2): we use **softplus** (or **tanh**), never ReLU.

Training. We minimize a one-step loss on $(\mathbf{q}, \dot{\mathbf{q}}, \boldsymbol{\tau}, \ddot{\mathbf{q}})$ transitions combining the forward-dynamics residual (predicted vs. measured $\ddot{\mathbf{q}}$) and the inverse-dynamics residual (predicted vs. measured $\boldsymbol{\tau}$), with input normalization fit on the training set.

4 Statistical target and finite-horizon guarantee

The statistical target is the held-out *one-step acceleration MSE*

$$\varepsilon_{\text{acc}}(\hat{f}; \rho) := \mathbb{E}_{(\mathbf{x}, \boldsymbol{\tau}) \sim \rho} \left[\left\| \hat{f}(\mathbf{x}, \boldsymbol{\tau}) - f^*(\mathbf{x}, \boldsymbol{\tau}) \right\|_2^2 \right], \quad \mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}}). \quad (6)$$

We write $\delta_{\text{acc}} := \sqrt{\varepsilon_{\text{acc}}}$ for its root mean-squared error. The H -step rollout MSE remains an important diagnostic, but it is not the statistical primitive in the bound: it already contains closed-loop distribution shift and compounding error, which the simulation argument below handles explicitly.

The code uses semi-implicit Euler with step Δt :

$$\dot{\mathbf{q}}^+ = \dot{\mathbf{q}} + \Delta t f(\mathbf{x}, \boldsymbol{\tau}), \quad \mathbf{q}^+ = \mathbf{q} + \Delta t \dot{\mathbf{q}}^+.$$

Consequently an acceleration error $\mathbf{e} = \hat{f} - f^*$ induces the exact one-step state-transition error

$$\left\| \hat{F}(\mathbf{x}, \boldsymbol{\tau}) - F^*(\mathbf{x}, \boldsymbol{\tau}) \right\|_2 = c_{\Delta t} \|\mathbf{e}\|_2, \quad c_{\Delta t} := \Delta t \sqrt{1 + \Delta t^2}. \quad (7)$$

Assumption 1 (Coverage and Lipschitz values). *For every policy used by the optimality comparison and every time $t \in \{0, \dots, H-1\}$, the relevant state-action occupancy d_t^π is absolutely continuous with respect to the aggregated training distribution ρ , with density ratio at most C_{cov} . The continuation value at time $t+1$ is L_{t+1} -Lipschitz in the Euclidean state norm.*

Proposition 1 (MSE-to-control conversion). *Let $\hat{\pi}$ be ε_{opt} -optimal for the learned transition $\hat{F}$, and let π^* be optimal for F^* . Under Assumption 1,*

$$J_{F^*}(\pi^*) - J_{F^*}(\hat{\pi}) \leq 2c_{\Delta t} \left(\sum_{t=0}^{H-1} L_{t+1} \right) \sqrt{C_{\text{cov}} \varepsilon_{\text{acc}}(\hat{f}; \rho)} + \varepsilon_{\text{opt}}. \quad (8)$$

Proof sketch. Apply the finite-horizon value-difference identity to a fixed policy and bound each transition perturbation by the Lipschitz constant of the continuation value. Change measure from d_t^π to ρ using the density-ratio bound, then apply Cauchy–Schwarz and (7). Applying the fixed-policy bound once to π^* and once to $\hat{\pi}$, and inserting the learned-model planning error, gives (8). If $L_{t+1} \leq L_V(H - t - 1)$, then the leading constant is at most $c_{\Delta t} L_V H(H - 1)$, displaying the usual $O(H^2)$ amplification.

Resolution of the square root. Equation (8) is proportional to $\sqrt{\varepsilon_{\text{acc}}}$ because (6) is a *squared* error. The same bound is linear in the RMSE δ_{acc} . We therefore report MSE in experiments, take its square root only when inserting it into a control-performance bound, and never use the same symbol for both quantities.

4.1 On-policy data are not treated as i.i.d.

Data are collected in rounds. Conditional on the history before round m , the policy π_m is frozen and its Markov chain is assumed stationary after burn-in and geometrically β -mixing:

$$\beta_m(k) \leq B_m \exp(-k/\tau_m).$$

For a block length b , alternating blocks yield $N_{\text{eff}} = \lfloor N/(2b) \rfloor$ approximately independent blocks; b is chosen so the coupling remainder $2N_{\text{eff}}\beta_m(b)$ is below the allocated failure probability. Supervised generalization is applied conditionally within each round and then union-bounded over rounds. The training distribution is the data-aggregation mixture $\rho = \sum_m \alpha_m d^{\pi_m}$. Assumption 1 is still required for the final policy; without it, the theorem makes no claim about unvisited regions.

Claim scope. The paper’s current claim is a *conditional improvement of an upper bound*: under realizability, coverage, mixing, and comparable optimization, the structured class can have a smaller estimation-complexity term. This is not yet a two-sided separation. A separation would additionally require a matching lower bound for the unstructured class, which is outside the present result and will not be implied by empirical curves alone.

4.2 A concrete, executable complexity proxy

For d generalized coordinates, a dense mass-matrix head emits d^2 functions, while the Cholesky head emits only $d(d + 1)/2$. Including the scalar potential head gives the output-coordinate ratio

$$\kappa_{\text{out}} := \frac{d^2 + 1}{d(d + 1)/2 + 1}. \quad (9)$$

For the Franka ($d = 7$), the executable count is 49 versus 28 mass entries, hence $\kappa_{\text{out}} = 1.72$.

Output coordinates alone overstate the gain when hidden layers dominate. Using the exact MLP parameter-count formula

$$P(n_0, \dots, n_L) = \sum_{\ell=1}^L (n_{\ell-1} + 1)n_\ell,$$

the repository’s two [128, 128] energy heads contain $P_{\text{DeLaN}} = 38,813$ trainable parameters. Replacing only the Cholesky head by a dense $d \times d$ mass head gives $P_{\text{full-energy}} = 41,522$, so the portion isolated to Cholesky is

$$\kappa_{\text{chol,param}} := P_{\text{full-energy}}/P_{\text{DeLaN}} = 1.07.$$

This modest value is the honest architecture-level effect of Cholesky alone.

For the configured direct deterministic MLP, the exact count is $P_{\text{MLP}} = 139,015$, yielding $\kappa_{\text{direct}} = 3.58$. The five-member probabilistic ensemble has $P_{\text{ens}} = 704,140$ and $\kappa_{\text{ens}} = 18.14$. These latter ratios include the chosen widths and ensemble size; they are not intrinsic consequences of Lagrangian mechanics.

To connect the count to a sample upper bound, suppose the class-specific estimation term has the standard form

$$\varepsilon_{\text{acc}}(\hat{f}_{\mathcal{H}}; \rho) \leq \varepsilon_{\text{approx}}(\mathcal{H}) + A_{\mathcal{H}} \sqrt{\frac{P_{\mathcal{H}} \log(cN_{\text{eff}}) + \log(1/\delta)}{N_{\text{eff}}}}.$$

If approximation errors and constants $A_{\mathcal{H}}$ are controlled, the certified samples needed for a fixed estimation tolerance scale linearly in $P_{\mathcal{H}}$; the ratio of those *upper-bound requirements* is the corresponding $\kappa = P_{\mathcal{U}}/P_{\mathcal{S}}$. This remains a conditional upper-bound comparison, not a lower bound on the MLP.

5 Model architectures (what we use in each case)

Role	Model	Architecture
Structured (<i>proposed</i>)	DeLaN [1]	energy heads $\mathbf{L}(\mathbf{q}), V(\mathbf{q})$, MLP [128, 128], softplus (C^2), SPD via (4)
Unstructured baseline	MLP ensemble (PETS) [5]	$K=5$ probabilistic MLPs [256, 256, 256], SiLU, Gaussian $\ddot{\mathbf{q}}$ head
Minimal reference	MLP	single deterministic MLP [256, 256, 256], SiLU

Table 1: Models compared. The structured model constrains the hypothesis class to physically consistent dynamics; the ensemble is the strong unstructured baseline and supplies epistemic uncertainty for planning. Widths/activations are the defaults; the benchmark tunes them per model.

Related structured models (future ablations). Lagrangian Neural Networks [2] learn an *arbitrary* $\mathcal{L}$ (dropping the $T = \frac{1}{2}\dot{\mathbf{q}}^\top \mathbf{M}\dot{\mathbf{q}}$ assumption); Hamiltonian Neural Networks [3] parameterize the Hamiltonian instead; and Lagrangian Graph Neural Networks [4] exploit the manipulator’s kinematic topology and scale to many links. They slot into the same model registry as drop-in alternatives to DeLaN.

6 The controller: planning through the learned model

Given the learned forward model $\hat{f}_{\theta} : (\mathbf{q}, \dot{\mathbf{q}}, \boldsymbol{\tau}) \mapsto \ddot{\mathbf{q}}$ and a one-step integrator $\hat{F}$ producing $(\mathbf{q}_{t+1}, \dot{\mathbf{q}}_{t+1})$, control is *model-predictive*. At each state $\mathbf{x}_t = (\mathbf{q}_t, \dot{\mathbf{q}}_t)$ we optimize a torque sequence over a horizon H ,

$$\min_{\boldsymbol{\tau}_{t:t+H-1}} \sum_{k=0}^{H-1} \ell(\mathbf{x}_{t+k}, \boldsymbol{\tau}_{t+k}) + \ell_f(\mathbf{x}_{t+H}), \quad \mathbf{x}_{t+k+1} = \hat{F}(\mathbf{x}_{t+k}, \boldsymbol{\tau}_{t+k}), \quad (10)$$

apply the first torque, and re-plan (receding horizon). The default solver is **MPPI** [7]: sample N noisy control sequences, roll each through $\hat{F}$, and update the control by the cost-weighted average

$$\boldsymbol{\tau} \leftarrow \sum_{i=1}^N w_i \boldsymbol{\tau}^{(i)}, \quad w_i \propto \exp\left(-\frac{1}{\lambda} (J^{(i)} - \min_j J^{(j)})\right), \quad (11)$$

with $J^{(i)}$ the rolled-out cost (10) and temperature λ . MPPI needs *no policy network*, so its performance is governed almost entirely by the accuracy of $\hat{f}_{\boldsymbol{\theta}}$ —this is the cleanest probe of whether the Lagrangian prior helps. The reach cost is the end-effector position (and orientation) error to the commanded pose plus a torque-effort penalty.

Alternatives. A Dyna/MBPO variant [6] would instead generate short model rollouts and run an off-policy update (e.g. SAC) on mixed real+model data; a model-free PPO policy serves as the lower-bound baseline. These are deferred until MPPI results justify a learned policy.

7 Evaluation protocol

Models are compared on held-out one-step acceleration MSE (and, for the structured model, energy drift and $\lambda_{\min}(\mathbf{M})$), under matched data: the same seeds and the same per-seed transitions feed every model. We tune hyper-parameters per model, retrain the winner over ≥ 5 seeds, and report the mean with a percentile-bootstrap 95% confidence interval. The data-efficiency story is the curve of model error (and, later, task return) versus the number of real transitions N .

References

- [1] M. Lutter, C. Ritter, J. Peters. *Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning*. ICLR, 2019.
- [2] M. Cranmer et al. *Lagrangian Neural Networks*. ICLR Deep Differential Equations Workshop, 2020.
- [3] S. Greydanus, M. Dzamba, J. Yosinski. *Hamiltonian Neural Networks*. NeurIPS, 2019.
- [4] R. Bhattoo, S. Ranu, N. M. A. Krishnan. *Learning Articulated Rigid Body Dynamics with Lagrangian Graph Neural Network*. NeurIPS, 2022.
- [5] K. Chua, R. Calandra, R. McAllister, S. Levine. *Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models (PETS)*. NeurIPS, 2018.
- [6] M. Janner, J. Fu, M. Zhang, S. Levine. *When to Trust Your Model: Model-Based Policy Optimization (MBPO)*. NeurIPS, 2019.
- [7] G. Williams, A. Aldrich, E. A. Theodorou. *Model Predictive Path Integral Control*. Journal of Guidance, Control, and Dynamics, 2017.
- [8] A. Agarwal, N. Jiang, S. M. Kakade, W. Sun. *Reinforcement Learning: Theory and Algorithms*. Online monograph, 2022.
- [9] M. Mohri, A. Rostamizadeh. *Stability Bounds for Stationary ϕ -mixing and β -mixing Processes*. Journal of Machine Learning Research, 2010.